



UNIVERSITY OF
DERBY

Module 6CM521: Frontiers in AI and Data Science



Reinforcement Learning

Ioannis Tsioulis (MC)



derby.ac.uk

Slides acknowledgements:
Dr Hong Qing Yu

Acknowledgement

Most of the slides are adapted from the lecture slides of Dr Hong Qing Yu (Harry).

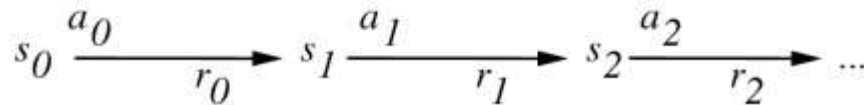
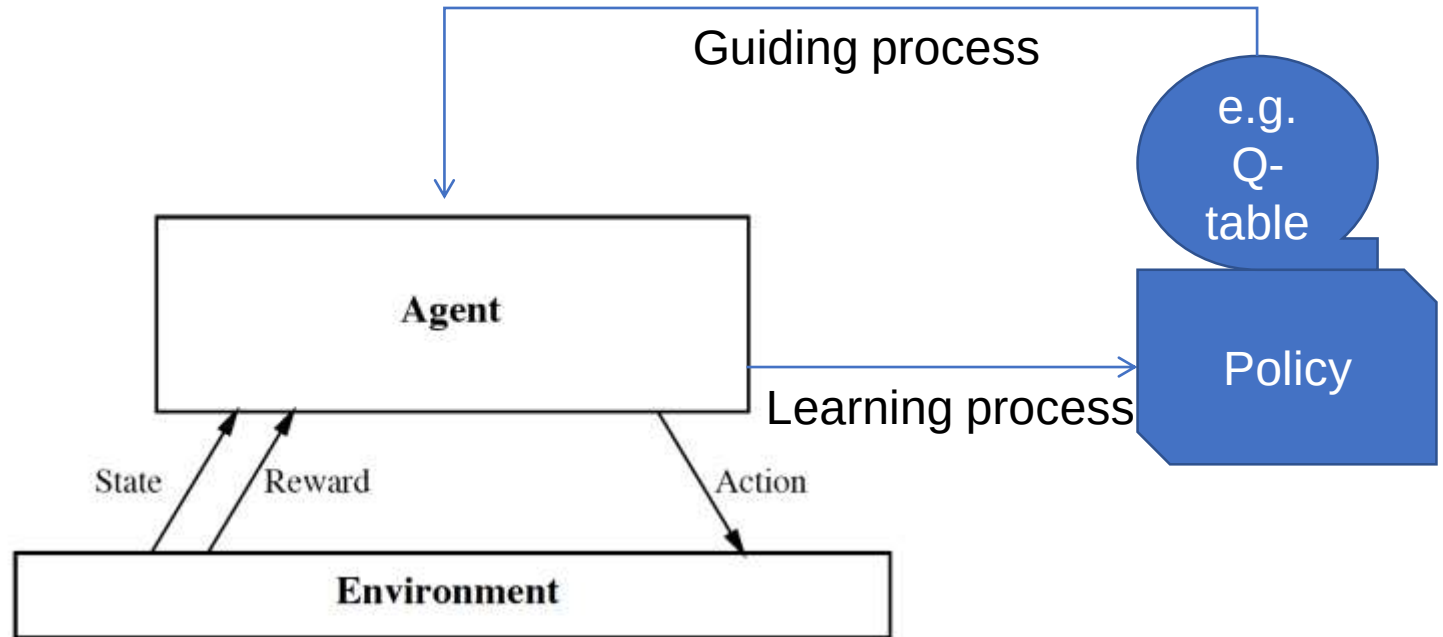
Overview

- Supervised Learning: Immediate feedback (labels provided for every input).
- Unsupervised Learning: No feedback (no labels provided).
- Reinforcement Learning (RL): Delayed scalar feedback (a number called reward).
- RL deals with agents that must sense & act upon their environment. This is combines classical AI and machine learning techniques. It the most comprehensive problem setting.
- Examples:
 - A robot cleaning my room and recharging its battery
 - Robot-soccer
 - How to invest in shares
 - Modeling the economy through rational agents
 - Learning how to fly a helicopter
 - Scheduling planes to their destinations
 - and so on

Complications

- The outcome of your actions may be uncertain
- You may not be able to perfectly sense the state of the world
- The reward may be stochastic.
- Reward is delayed (i.e. finding food in a maze)
- You may have no clue (model) about how the world responds to your actions.
- You may have no clue (model) of how rewards are being paid off.
- The world may change while you try to learn it
- How much time do you need to explore uncharted territory before you exploit what you have learned?

The Big Picture



Your action influences the state of the world which determines its reward

The Task

- To learn an optimal *policy* that maps states of the world to actions of the agent.

I.e., if this patch of room is dirty, I clean it. If my battery is empty, I recharge it.


$$\pi : \mathcal{S} \rightarrow \mathcal{A}$$

- What is it that the agent tries to optimize?

Answer: the **total future discounted reward**:

$$\begin{aligned} V^{\pi}(s_t) &= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \\ &= \sum_{i=0}^{\infty} \gamma^i r_{t+i} \quad 0 \leq \gamma < 1 \end{aligned}$$

Note: immediate reward is worth more than future reward.

Value Function

- Let's say we have access to the optimal value function that computes the total future discounted reward.

$$V^*(s)$$

- What would be the optimal policy ?

$$\pi^*(s)$$

- Answer: we choose the action that maximizes:

$$p^*(s) = \underset{a}{\operatorname{argmax}} \left(\underset{a}{\mathbb{E}} [r(s,a) + \gamma V^*(\delta(s,a))] \right)$$

- We assume that we know what the reward will be if we perform action “a” in state “s”:

$$r(s,a)$$

- We also assume we know what the next state of the world will be if we perform action “a” in state “s”:

$$s_{t+1} = \delta(s_t, a)$$

How do we structure Reinforcement Learning problem formally?

- MDP: Markov Decision Process (MDP) - Structuring A Reinforcement Learning Problem
- MDP gives us a way to formalize sequential decision making. This formalization is the basis for structuring problems that are solved with reinforcement learning.

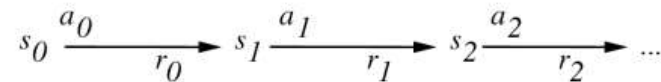
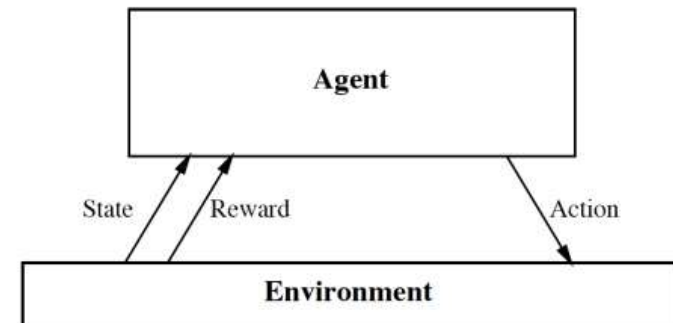


Components Of An MDP

In an MDP, we have a decision maker, called **an agent**, that interacts with the **environment** it's placed in. These interactions occur sequentially **over time**. At each time step, the agent will get some representation of the environment's **state**. Given this representation, the agent selects an **action** to take. The environment is then transitioned into a new state, and the agent is given a **reward** as a consequence of the previous action.

Components of an MDP over time:

- Agent
- Environment
- State
- Action
- Reward



MDP Notation

We're now going to repeat what we just casually discussed but in a more formal and mathematically notated way.

In an MDP, we have a set of states $\mathbf{S}$, a set of actions $\mathbf{A}$, and a set of rewards $\mathbf{R}$. We'll assume that each of these sets has a finite number of elements.

At each time step $t = 0, 1, 2, \dots$, the agent receives some representation of the environment's state $S_t \in \mathbf{S}$. Based on this state, the agent selects an action $A_t \in \mathbf{A}$. This gives us the state-action pair (S_t, A_t) .

Time is then incremented to the next time step $t + 1$, and the environment is transitioned to a new state $S_{t+1} \in \mathbf{S}$. At this time, the agent receives a numerical reward $R_{t+1} \in \mathbf{R}$ for the action A_t taken from state S_t .

We can think of the process of receiving a reward as an arbitrary function f that maps state-action pairs to rewards. At each time t , we have

$$f(S_t, A_t) = R_{t+1}.$$

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$

Transition Probabilities

Since the sets $\mathbf{S}$ and $\mathbf{R}$ are finite, the **random variables** R_t and S_t have well defined probability distributions. In other words, all the possible values that can be assigned to R_t and S_t have some associated probability. These distributions depend on the *preceding* state and action that occurred in the previous time step $t - 1$.

For example, suppose $s' \in \mathbf{S}$ and $r \in \mathbf{R}$. Then there is *some* probability that $S_t = s'$ and $R_t = r$. This probability is determined by the particular values of the *preceding* state $s \in \mathbf{S}$ and action $a \in \mathbf{A}(s)$. Note that $\mathbf{A}(s)$ is the set of actions that can be taken from state s .

Let's define this probability.

For all $s' \in \mathbf{S}$, $s \in \mathbf{S}$, $r \in \mathbf{R}$, and $a \in \mathbf{A}(s)$, we define the probability of the transition to state s' with reward r from taking action a in state s as:

$$p(s', r \mid s, a) = \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}.$$

Expected Return - What Drives A Reinforcement Learning Agent In An MDP

As we stated that the goal of an agent in an MDP is to maximize its cumulative rewards. We need a way to aggregate and formalize these cumulative rewards. For this, we introduce the concept of the *expected return* of the rewards at a given time step.

For now, we can think of the return simply as the sum of future rewards. Mathematically, we define the return $\mathbf{G}$ at time $\mathbf{t}$ as

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T,$$

where T is the final time step.

Think about: Episodic Vs. Continuing Tasks ($T = \infty$)

Discounted Return

To define the discounted return, we first define the discount rate, γ , to be a number between 0 and 1. The discount rate will be the rate for which we discount future rewards and will determine the present value of future rewards. With this, we define the *discounted return* as

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \\ &= \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \end{aligned}$$

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

If the reward at each time step is a constant 1, immediate reward is 0 and $\gamma < 1$, then the return is

$$G_t = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma}. \quad (\text{Infinite Geometric Series})$$

How Good Is A State Or Action?

- First, we'd probably like to know how likely it is for an agent to take any given action from any given state. In other words, what is the probability that an agent will select a specific action from a specific state? This is where the notion of *policies* come into play, and we'll expand on this in just a moment.
- Secondly, in addition to understanding the probability of selecting an action, we'd probably also like to know how good a given action or a given state is for the agent. In terms of rewards, selecting one action over another in a given state may increase or decrease the agent's rewards, so knowing this in advance will probably help our agent out with deciding which actions to take in which states. This is where *value functions* become useful, and we'll also expand on this idea in just a bit.

Policies

- A policy is a function that maps a given state to probabilities of selecting each possible action from that state. We will use the symbol π to denote a policy.
- When speaking about policies, formally we say that an agent “follows a policy.” For example, if an agent follows policy π at time t , then $\pi(a|s)$ is the probability that $A_t=a$ if $S_t=s$. This means that, at time t , under policy π , the probability of taking action a in state s is $\pi(a|s)$.
- Note that, for each state $s \in S$, π is a probability distribution over $a \in A(s)$.

Value Functions following Policy

- State-Value Function

Formally, the value of state s under policy π is the expected return from starting from state s at time t and following policy π thereafter. Mathematically we define $v_\pi(s)$ as

$$\begin{aligned} v_\pi(s) &= E_\pi[G_t \mid S_t = s] \\ &= E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s\right]. \end{aligned}$$

- Action-Value Function

Formally, the value of action a in state s under policy π is the expected return from starting from state s at time t , taking action a , and following policy π thereafter. Mathematically, we define $q_\pi(s, a)$ as

$$\begin{aligned} q_\pi(s, a) &= E_\pi[G_t \mid S_t = s, A_t = a] \\ &= E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right]. \end{aligned}$$

What Do Reinforcement Learning Algorithms Learn?

It is the goal of reinforcement learning algorithms to find a policy that will yield **a lot of rewards** for the agent if the agent indeed **follows that policy**. Specifically, reinforcement learning algorithms seek to **find a policy that will yield more return** to the agent **than all other policies**.

Optimal Policy

In terms of return, a policy π is considered to be better than or the same as policy π' if the expected return of π is greater than or equal to the expected return of π' for all states. In other words,

$$\pi \geq \pi' \text{ if and only if } v_{\pi}(s) \geq v_{\pi'}(s) \text{ for all } s \in S.$$

Remember, $v_{\pi}(s)$ gives the expected return for starting in state s and following π thereafter. A policy that is better than or at least the same as all other policies is called the *optimal policy*.

Optimal Functions

Optimal State-Value Function

The optimal policy has an associated *optimal* state-value function. Recall, we covered state-value functions in detail **last time**. We denote the optimal state-value function as v_* and define as

$$v_*(s) = \max_{\pi} v_{\pi}(s)$$

for all $s \in \mathcal{S}$. In other words, v_* gives the largest expected return achievable by any policy π for each state.

Optimal Action-Value Function

Similarly, the optimal policy has an *optimal* action-value function, or *optimal* Q-function, which we denote as q_* and define as

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$. In other words, q_* gives the largest expected return achievable by any policy π for each possible state-action pair.

Q-Function (Bellman Optimality Equation for Q_*)

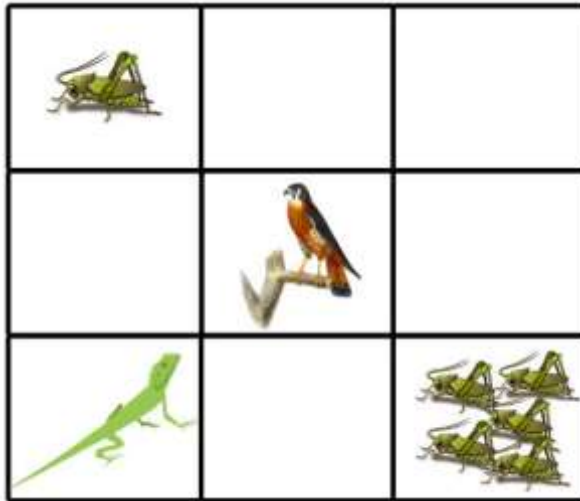
One fundamental property of q_* is that it must satisfy the following equation.

$$q_*(s, a) = E \left[R_{t+1} + \gamma \max_{a'} q_*(s', a') \right]$$

This is called the *Bellman optimality equation*. It states that, for any state-action pair (s, a) at time t , the expected return from starting in state s , selecting action a and following the optimal policy thereafter (AKA *the Q-value* of this pair) is going to be the expected reward we get from taking action a in state s , which is R_{t+1} , plus the *maximum* expected discounted return that can be achieved from any possible next state-action pair (s', a') .

Since the agent is following an optimal policy, the following state s' will be the state from which the best possible next action a' can be taken at time $t + 1$.

Q-Learning example



Finite MDP and reward

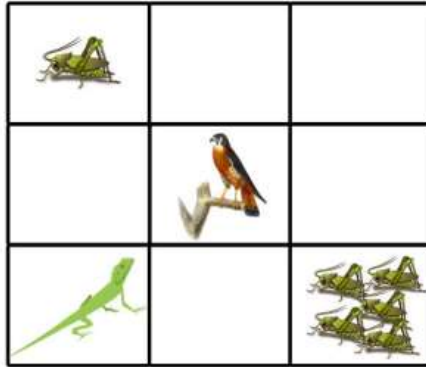
State	Reward
One cricket	+1
Empty	-1
Five crickets	+10 Game over
Bird	-10 Game over

A:    
right down up left

Now, at the start of the game, the lizard has no idea how good any given action is from any given state. It's not aware of anything besides the current state of the environment.

Therefore, the **Q-values** for each state-action pair will all be initialized to zero since the lizard knows nothing about the environment at the start. Throughout the game learning, though, the Q-values will be iteratively updated using value iteration.

Storing Q-Values In A Q-Table



Episodes

Let's say we want the lizard to play five episodes. It is during these episodes that the learning process will take place. In each episode, the lizard starts out by choosing an action from the starting state based on the current Q-values in the table. The lizard chooses the action based on which action has the highest Q-value in the Q-table for the current state.

At the start



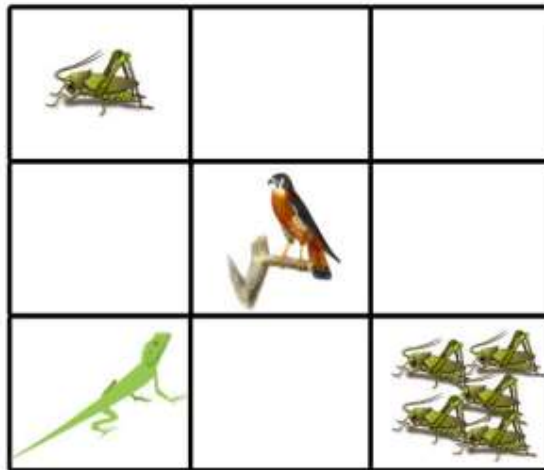
	Actions			
	Left	Right	Up	Down
1 cricket	0	0	0	0
Empty 1	0	0	0	0
Empty 2	0	0	0	0
Empty 3	0	0	0	0
Bird	0	0	0	0
Empty 4	0	0	0	0
Empty 5	0	0	0	0
Empty 6	0	0	0	0
5 crickets	0	0	0	0

States

But, wait... That's kind of weird for the first actions in the first episode, right?

Exploration Vs. Exploitation

- *Exploration* is the act of exploring the environment to find out information about it.
- *Exploitation* is the act of exploiting the information that is already known about the environment in order to maximize the return.



Think of our game. If our lizard got to the single cricket before it got to the group of five crickets, then only making use of exploitation, going forward the lizard would just learn to exploit the information it knows about the location of the single cricket to get single incremental points infinitely. It would then also be losing single points infinitely just to back out of the tile before it can come back in to get the cricket again.

Exploration Vs. Exploitation (Cont.)

- Epsilon Greedy Strategy

With this strategy, we define an *exploration rate* ϵ that we initially set to 1. This exploration rate is the probability that our agent will explore the environment rather than exploit it. With $\epsilon=1$, it is 100% certain that the agent will start out by exploring the environment.

As the agent learns more about the environment, at the start of each new episode, ϵ will decay by some rate that we set so that the likelihood of exploration becomes less and less probable as the agent learns more and more about the environment. The agent will become “*greedy*” in terms of exploiting the environment once it has had the opportunity to explore and learn more about it.

Update the Q-Table

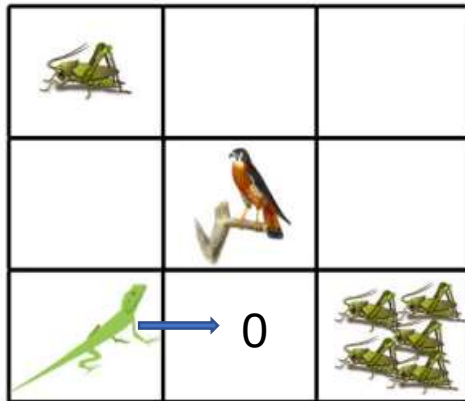
- To update the Q-value for the action of moving right taken from the previous state, we use the Bellman equation.
- We want to make the Q-value for the given state-action pair as close as we can to the right hand side of the Bellman equation so that the Q-value will eventually converge to the optimal Q-value.

The formula for calculating the new Q-value for state-action pair (s, a) at time t is this:

$$q^{new}(s, a) = (1 - \alpha) \underbrace{q(s, a)}_{\text{old value}} + \underbrace{\alpha \left(R_{t+1} + \gamma \max_{a'} q(s', a') \right)}_{\text{learned value}}$$

Learning rate

Update example



All of the math for this calculation of our concrete example state-action pair of moving right from the starting state is shown below. Suppose the discount rate $\gamma = 0.99$. We have

$$q^{new}(s, a) = (1 - \alpha) \underbrace{q(s, a)}_{\text{old value}} + \alpha \overbrace{\left(R_{t+1} + \gamma \max_{a'} q(s', a') \right)}^{\text{new value}}$$

$$= (1 - 0.7)(0) + 0.7 \left(-1 + 0.99 \left(\max_{a'} q(s', a') \right) \right)$$

$$\max_{a'} q(s', a') = \max(q(\text{empty6, left}), q(\text{empty6, right}), q(\text{empty6, up}), q(\text{empty6, down}))$$

$$= \max(0, 0, 0, 0)$$

$$= 0$$

$$q^{new}(s, a) = (1 - \alpha) \underbrace{q(s, a)}_{\text{old value}} + \alpha \overbrace{\left(R_{t+1} + \gamma \max_{a'} q(s', a') \right)}^{\text{new value}}$$

$$= (1 - 0.7)(0) + 0.7 \left(-1 + 0.99 \left(\max_{a'} q(s', a') \right) \right)$$

$$= (1 - 0.7)(0) + 0.7(-1 + 0.99(0))$$

$$= 0 + 0.7(-1)$$

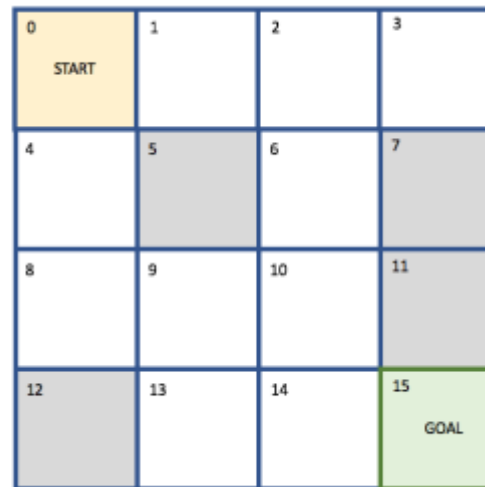
$$= -0.7$$

	Left	Right	Up	Down
1 cricket	0	0	0	0
Empty 1	0	0	0	0
Empty 2	0	0	0	0
Empty 3	0	0	0	0
Bird	0	0	0	0
Empty 4	0	0	0	0
Empty 5	0	0	0	0
Empty 6	0	0	0	0
5 crickets	0	0	0	0

Code example

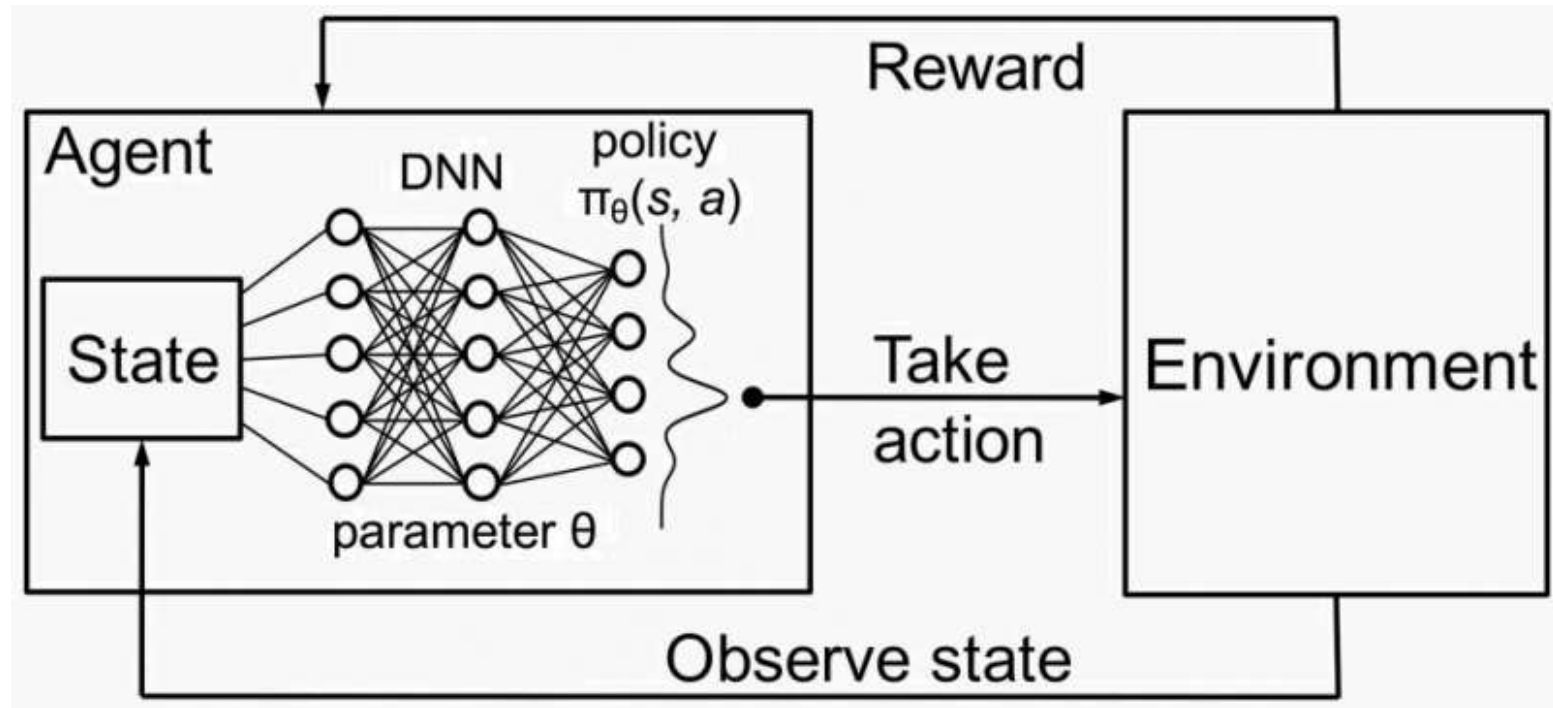
OpenAI Gym has a ton of environments ranging from simple text based games to Atari games like Breakout and Space Invaders.

Frozen Lake



State	Description	Reward
S	Agent's starting point - safe	0
F	Frozen surface - safe	0
H	Hole - game over	0
G	Goal - game over	1

Combining Reinforcement learning with Deep Learning



Conclusion

- Reinforcement learning addresses a very broad and relevant question: How can we learn to survive in our environment?
- We have looked at Q-learning, which simply learns from experience. No model of the world is needed.
- We made simplifying assumptions: e.g. state of the world only depends on last state and action. This is the *Markov* assumption. The model is called a *Markov Decision Process (MDP)*.
- We assumed deterministic dynamics, reward function, but the world really is stochastic.
- There are many extensions to speed up learning.
- There have been many successful real world applications.

http://elsy.gdan.pl/index.php?option=com_content&task=view&id=20&Itemid=39